CHAPTER 14

Socket API & UDP

socket · bind · listen · accept · connect · send/recv · TCP vs UDP calls · 4-tuple · concurrent servers

QUICK TRICK

TCP server: **socket → bind → listen → accept → recv/send → close**. TCP client: **socket → connect → send/recv → close**. **connect()** sends the SYN. **listen()** makes the socket passive. **accept()** returns a **new** socket. UDP: **sendto / recvfrom**, no listen or accept.

14.1 What is a socket?

● **LEVEL 1 · BEGINNER**

A **socket** is the doorway between an application and the transport layer. It is identified by an **(IP address, port)** pair. The socket API is the set of calls (socket, bind, listen, accept, connect, send, recv, close) that programs use. Two types: **SOCK_STREAM** (TCP, reliable byte stream) and **SOCK_DGRAM** (UDP, independent datagrams).

Analogy. A telephone. *socket()* buys the phone, *bind()* gives it a number, *listen()* switches the ringer on, *accept()* picks up when it rings, *connect()* dials.

14.2 TCP call sequence

● **LEVEL 2 · EXAM**

```
        SERVER                                        CLIENT
socket()   create socket                       socket()   create socket
bind()     attach to (IP, port)                (bind optional: OS picks an ephemeral port)
listen()   passive socket, backlog queue
accept()   BLOCKS until a connection  <=====   connect()  sends SYN, blocks until ESTABLISHED
           returns a NEW socket                (SYN / SYN+ACK / ACK happen inside the kernel)
recv()/send()  <---------- data ---------->    send()/recv()
close()                                        close()    (FIN exchange)
```

Call	What it does	Blocks?
socket()	Creates an endpoint; returns a descriptor	No
bind()	Assigns local IP and port	No
listen()	Marks the socket **passive**; sets the queue (backlog) of pending connections	No
accept()	Takes one completed connection from the queue; **returns a new connected socket**; the listening socket keeps listening	Yes
connect()	Starts the 3-way handshake (**sends SYN**)	Yes, until established
send() / recv()	Transfer bytes	recv blocks when empty
close()	Starts the FIN exchange	No

QUICK TRICK

The server finishes the handshake in the **kernel** while it is in listen. accept() then simply hands over a connection that is already ESTABLISHED.

14.3 Ports, connections and servers

● **LEVEL 2 · EXAM**

- A connection is identified by the **4-tuple** (src IP, src port, dst IP, dst port). Many clients can connect to the **same server port**, because their tuples differ.
- A server with **1 listening socket and n connected clients** has **n + 1** sockets.

AMJESH (https://singhamjesh.github.io/)

- The client usually gets an **ephemeral port** (about 49152–65535) automatically at connect().
- **Iterative server:** serves one client at a time. **Concurrent server:** after accept(), creates a process or thread for each connection.
- TCP is a **byte stream**: three send() calls of 100 B may arrive as one recv() of 300 B. Message boundaries are **not** kept.

14.4 UDP and its calls

● LEVEL 1 · BEGINNER

```
UDP server:  socket(SOCK_DGRAM) -> bind() -> recvfrom() -> sendto() -> ...
UDP client:  socket(SOCK_DGRAM) -> sendto() -> recvfrom()
No listen(), no accept(), no handshake. Each datagram carries the sender address.
```

	TCP	UDP
Connection	Yes (handshake)	No
Reliability, order	Yes	No
Message boundaries	No (byte stream)	**Yes** (one sendto = one recvfrom)
Flow / congestion control	Yes	No
Header	20 B or more	8 B: src port, dst port, length, checksum
Typical use	Web, file transfer	DNS, streaming, games

14.5 Common mistakes

COMMON MISTAKE

- Saying **accept()** sends the SYN (it is **connect()**).
- Saying the listening socket is used for data (the **new** socket from accept() is).
- Using listen() or accept() in UDP programs.
- Believing one send() always equals one recv() with TCP.

QUICK TRICK

Memory line: "Client dials (connect), server answers (accept)."

GATE QUICK CHECK — Chapter 14

```
TCP server : socket, bind, listen, accept, recv/send, close
TCP client : socket, (bind), connect, send/recv, close         SYN is sent by connect()
accept()   : blocks, returns NEW socket       listen(): passive + backlog
UDP        : socket, (bind), sendto/recvfrom   no handshake      sockets on server = 1 + n clients
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Which call sends the SYN?

Q2. Which call makes a socket passive?

Q3. Which call returns a new socket descriptor?

Q4. Do UDP servers call listen() and accept()?

● LEVEL 2 · EXAM

Q5. A TCP server has one listening socket and three connected clients. How many sockets does it hold?

Q6. How can two clients connect to the same server port at once?

Q7. A client does 3 send() calls of 100 B each over TCP. Can the server's one recv() return 300 B?

Q8. Client does 3 sendto() of 100 B over UDP. How many recvfrom() calls are needed to read them all?

● LEVEL 3 · GATE

Q9. Order the calls for a TCP server and client.

Q10. When connect() returns on the client, what has happened on the wire? And accept() on the server?

Q11. Why does a concurrent TCP server need accept() to return a new socket?

Answers

Q	Solution
1	**connect()**.
2	**listen()**.
3	**accept()**.
4	**No.** They use bind() then recvfrom()/sendto().
5	**4** (1 listening + 3 connected).
6	Their **(IP, port)** pairs differ, so the 4-tuples are different.
7	**Yes**: TCP is a byte stream with no message boundaries.
8	**3**, each returns one datagram.
9	Server: socket, bind, listen, accept, recv/send, close. Client: socket, connect, send/recv, close.
10	The 3-way handshake is complete (SYN, SYN+ACK, ACK sent). accept() returns a connection that is already ESTABLISHED, from the backlog queue.
11	The original socket must keep **listening** for new clients while each new socket handles one client.

Next: Chapter 15 — DNS and HTTP.

AMJESH singhamjesh.github.io